SIMATS ENGINEERING

ASSESSMENT TOOL 2

Pseudocode Writing Task (Algorithm Design)

COURSE NAME: COMPUTER VISION WITH OPENCV

COURSE CODE: DSA02

COURSE OUTCOME COVERED

CO5: *Understand video processing - motion computation - 3D vision - geometry.* (BTL 6)

Assessment Tool Weightage: 20%

Pseudocode Writing Tasks: Design a clear and structured pseudocode for the given problem by organizing the solution into logical stages of a computer vision pipeline. Ensure proper use of control flow, modular steps, and justification of key operations to satisfy the given constraints.

Code of Conduct:

I N HARSHAVARDHAN (192324189) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: N HARSHAVARDHAN

Problem 1: Real-Time Face Detection and Recognition Pipeline

Problem Statement

A system must process a live video stream to capture frames from a camera, detect faces in each frame, recognize faces using a pre-trained dataset, and display bounding boxes with labels in real time.

Inputs, Outputs & Constraints

Inputs: Live camera video stream; Pre-trained face detector model (e.g., Haar Cascade / DNN); Pre-trained face recognizer model and labelled face dataset

Outputs: Live display window showing each frame with bounding boxes and identity labels drawn on detected faces

Constraints: Must operate continuously; Must minimize processing delay; Must handle multiple faces per frame

Pseudocode

```
BEGIN FaceDetectionRecognitionPipeline
  INITIALIZE camera capture device
  LOAD pre-trained face detector model
  LOAD pre-trained face recognizer model and label dataset
  IF camera fails to open THEN
    PRINT "Camera error" AND EXIT
  END IF

  WHILE camera is open DO
    frame = CAPTURE next frame
    IF frame is empty THEN
      CONTINUE
    END IF

    grayFrame = CONVERT frame TO grayscale       // reduces data for faster detection
    faces = DETECT_FACES(grayFrame)              // modular detection step

    FOR EACH face IN faces DO
      faceROI = EXTRACT_REGION(grayFrame, face.coordinates)
      (label, confidence) = RECOGNIZE_FACE(faceROI)   // modular recognition step

      IF confidence >= ACCEPT_THRESHOLD THEN
        name = GET_LABEL_NAME(label)
      ELSE
        name = "Unknown"
      END IF

      DRAW_BOUNDING_BOX(frame, face.coordinates)
      DRAW_LABEL(frame, name, face.coordinates)
```

```
    END FOR

    DISPLAY frame
    IF USER_PRESSED('q') THEN
      BREAK
    END IF
  END WHILE

  RELEASE camera
  CLOSE all display windows
END
```

Design Justification (Constraint Handling)

- Converting to grayscale before detection reduces computational load, helping minimize delay.
- Detection and recognition are separated into modular functions (DETECT_FACES, RECOGNIZE_FACE) so each stage can be optimized or swapped independently.
- The FOR EACH loop over 'faces' processes an arbitrary number of detections per frame, satisfying the multiple-faces constraint.
- The outer WHILE loop with a non-blocking key check keeps the pipeline running continuously until the user explicitly stops it.

Problem 2: OCR Pipeline for Document Processing

Problem Statement

A document processing system must read input images containing printed text, enhance image quality, segment text regions, and extract and display recognized text, operating over a batch of documents.

Inputs, Outputs & Constraints

Inputs: Folder path containing multiple document images (possibly noisy, low contrast)

Outputs: Extracted text displayed and saved to an output file for each input image

Constraints: Input images may be noisy and low contrast; Batch processing of multiple images is required

Pseudocode

```
BEGIN OCRPipeline
  imageList = LIST all image file paths in input folder

  FOR EACH imagePath IN imageList DO        // batch processing loop
    image = READ_IMAGE(imagePath)
    IF image is empty THEN
      LOG_ERROR(imagePath)
      CONTINUE
    END IF

    grayImage = CONVERT image TO grayscale
    denoisedImage = APPLY_DENOISE(grayImage)         // e.g., median/Gaussian filter
    enhancedImage = ENHANCE_CONTRAST(denoisedImage)    // e.g., CLAHE / histogram equalization
    binaryImage = ADAPTIVE_THRESHOLD(enhancedImage)

    textRegions = SEGMENT_TEXT_REGIONS(binaryImage)    // contour / connected-component analysis
    extractedText = ""

    FOR EACH region IN textRegions DO
      regionCrop = CROP(binaryImage, region)
      regionText = OCR_ENGINE(regionCrop)            // e.g., Tesseract
      extractedText = extractedText + regionText
    END FOR

    DISPLAY extractedText
    SAVE extractedText TO outputFile NAMED_AFTER(imagePath)
  END FOR
END
```

Design Justification (Constraint Handling)

- Denoising and contrast enhancement (CLAHE) directly target the noisy, low-contrast constraint before segmentation.
- Adaptive thresholding (rather than a single global threshold) copes better with uneven illumination common in scanned documents.
- The outer FOR loop over imageList satisfies the batch-processing requirement, iterating the same modular pipeline over every document.
- Segmentation and OCR extraction are kept as separate calls so the region-detection strategy can be changed without altering the OCR step.

Problem 3: Motion Detection and Tracking System

Problem Statement

A surveillance system must read video input, detect motion between consecutive frames, track moving objects, and highlight detected motion regions in near real time.

Inputs, Outputs & Constraints

Inputs: Continuous video stream from a surveillance camera

Outputs: Video display with bounding boxes highlighting regions of detected motion

Constraints: Should handle dynamic background; Must operate in near real-time

Pseudocode

```
BEGIN MotionDetectionTracking
  videoCapture = OPEN video source
  backgroundModel = INITIALIZE adaptive background subtractor  // handles dynamic background
  trackers = EMPTY list

  WHILE videoCapture is open DO
    frame = READ next frame
    IF frame is empty THEN
      BREAK
    END IF

    grayFrame = CONVERT frame TO grayscale
    blurredFrame = APPLY_GAUSSIAN_BLUR(grayFrame)     // suppress noise before differencing

    foregroundMask = backgroundModel.APPLY(blurredFrame)  // adapts to slow background change
    cleanMask = MORPHOLOGICAL_OPEN_CLOSE(foregroundMask)  // remove small noise blobs

    contours = FIND_CONTOURS(cleanMask)
    detections = EMPTY list

    FOR EACH contour IN contours DO
      IF AREA(contour) > MIN_AREA_THRESHOLD THEN
        box = BOUNDING_RECT(contour)
        ADD box TO detections
      END IF
    END FOR

    trackers = UPDATE_TRACKERS(trackers, detections)   // associate detections across frames

    FOR EACH tracker IN trackers DO
      DRAW_BOUNDING_BOX(frame, tracker.box)
      DRAW_ID(frame, tracker.id)
    END FOR

    DISPLAY frame
    IF USER_PRESSED('q') THEN
```

```
      BREAK
    END IF
  END WHILE

  RELEASE videoCapture
END
```

Design Justification (Constraint Handling)

- An adaptive background subtractor (e.g., MOG2) is used instead of plain frame differencing, so gradual lighting or background changes do not trigger false motion.
- Morphological open/close removes small spurious noise regions from the mask, improving robustness of the dynamic background.
- A single-pass WHILE loop with lightweight per-frame operations (blur, subtraction, contour extraction) keeps processing within a near real-time budget.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient